

UNIT-4

CHAPTER-1

TREES

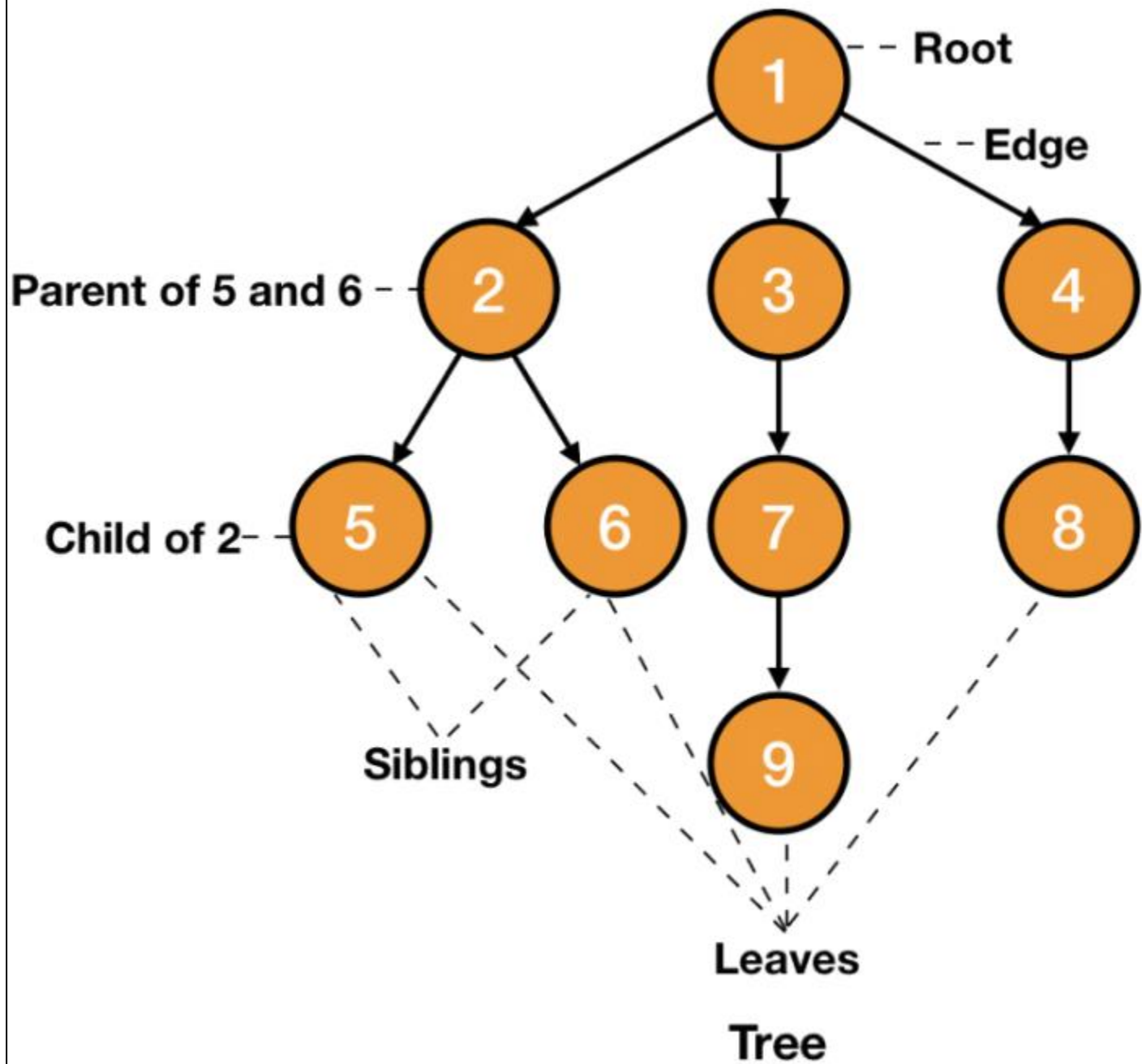
DATA STRUCTURE



Presented By:
Smitha Shetty
Lecturer in BCA Dept

Trees

- A tree is a nonlinear hierarchical data structure that consists of nodes connected by edges.
- A tree data structure is defined as a collection of objects or entities known as nodes that are linked together to represent a hierarchy.
- It's a non linear data structure as it does not store data in a sequential manner, but stores in a hierarchical fashion.



Tree Terminologies

- **Root:** The node which does not have any parent node.
- **Parent:** The node that is immediate predecessor of any node is known as a parent node.
- **Child:** The node that is immediate successor of any node is known as a child node.
- **Left Child:** The node which is on the left of the Binary Tree is called “Left-Child”
- **Right Child:** the node which is on the right of the binary tree is called “Right Child”.
- **Siblings:** Nodes having the same parent are sibling.
- **Descendent:** Any successor node on the path from that node to the leaf node.
- **Ancestor:** Any predecessor node on the path from root to that node.

Tree Terminologies

- **Level:** A level is the number of parent nodes corresponding to a given a node of the tree.
- **Edge:** The EDGE is the connecting line between any two nodes in a tree data structure
- **Path:** sequence of consecutive edges from source node to destination node.
- **Leaf Node:** Whenever a node in a tree data structure does not have any children, it is referred to as the LEAF Node.
- **Terminal Node:** The binary tree nodes with no successor are called as Terminal nodes.
- **Branch:** a path ending in a leaf
- **Depth of a node:** The depth of a node is the number of edges from root node to that node.

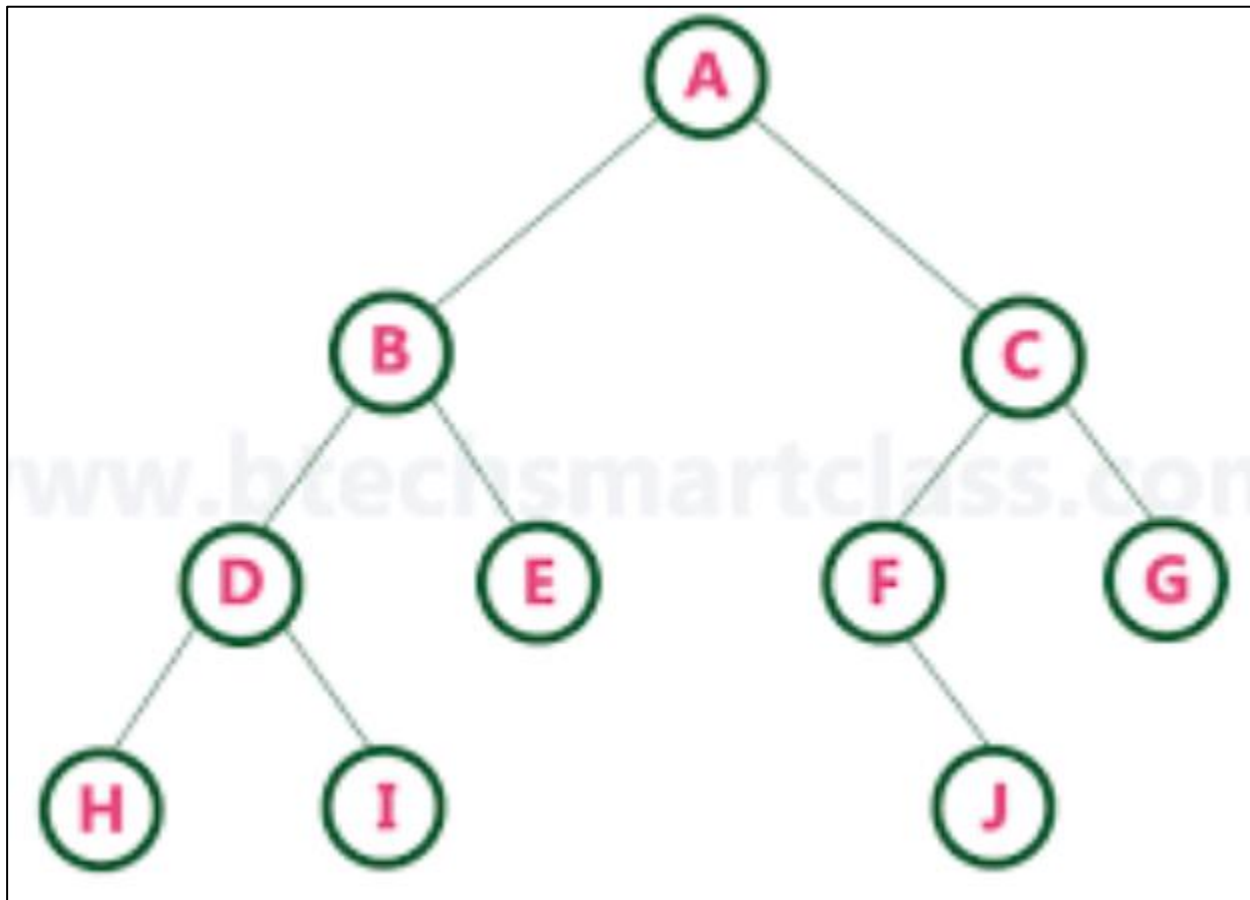
Tree Terminologies

- **Non Terminal Node:** nonterminal node of a tree. Any node that is not a terminal node (i.e. a leaf node) and hence has one or more children.
- **Degree of Node:** Degree of a node is the total number of children of that node.
- **Degree of a tree:** Degree of a tree is the highest degree of a node among all the nodes in the tree.
- **Subtree:** Subtree in any tree T , contains node with all its descendants.
- **Height of node:** Number of edges in the longest path from that node to a leaf.
- **Height of tree:** Number of edges in the longest path from root node to a leaf.

Binary Tree

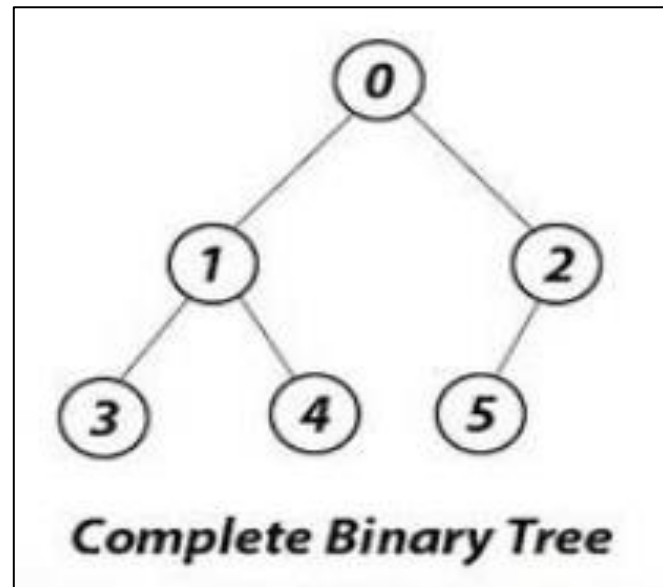
- A binary tree T is defined as a finite set of elements, called nodes, such that:
 - a) T is empty (called the null tree or empty tree), or
 - b) T contains a distinguished node R , called the root of T , and the remaining nodes of T form an ordered pair of disjoint binary trees T_1 and T_2 .

Binary Tree



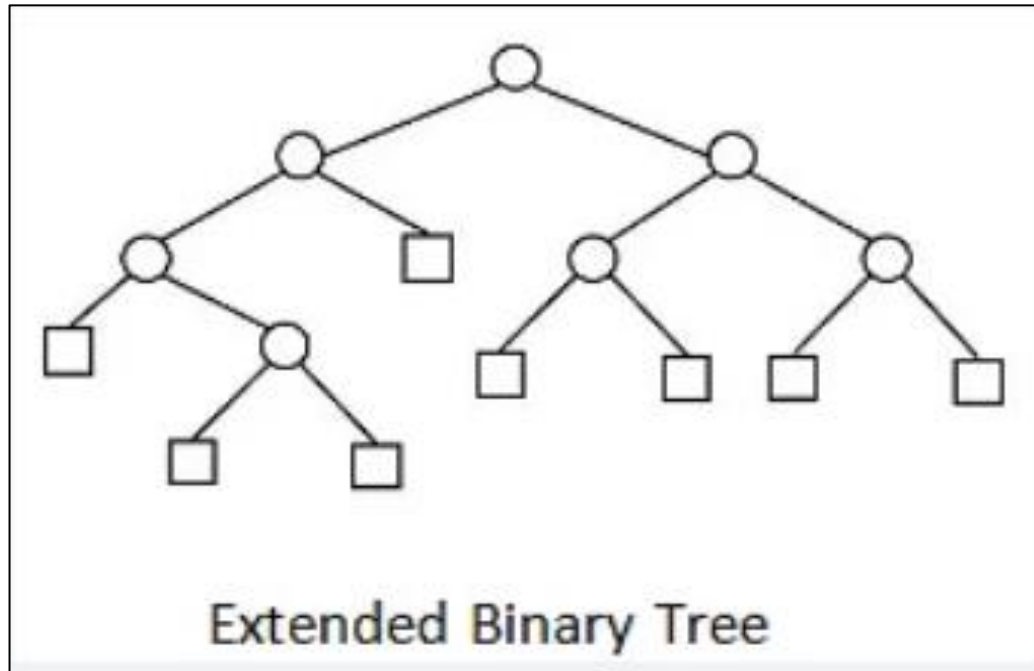
Complete Binary Tree

- A complete binary tree is a type of binary tree in which all the levels are completely filled (i.e., have two children), except possibly the deepest level. The deepest level can have nodes that can have either one left child or can be completely full.



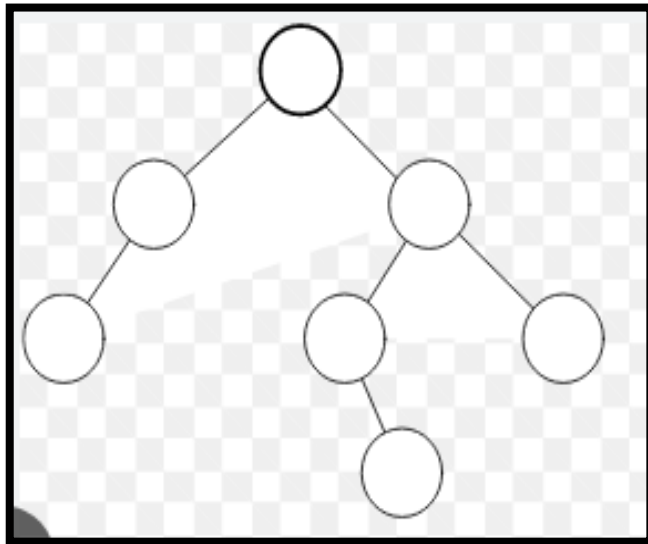
Extended Binary Tree

- A binary tree is said to be a 2-tree or an extended binary tree if each node N has either 0 or 2 children.

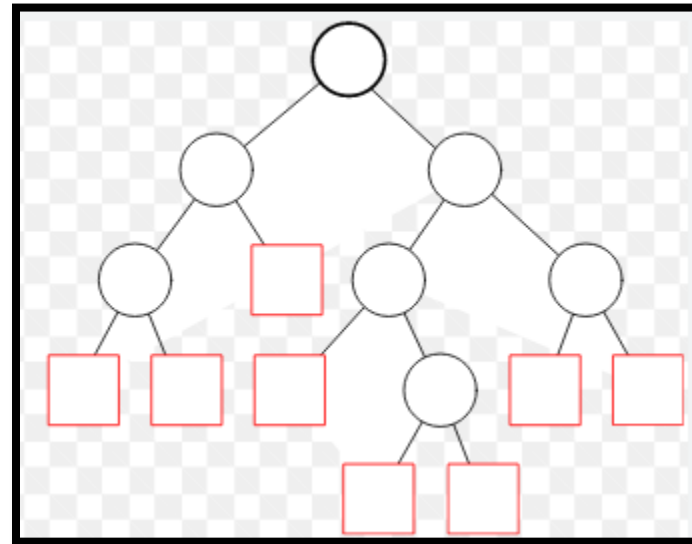


Internal Nodes and External Nodes

- The nodes with children are called Internal Nodes.
- The node with zero children are called External Nodes.



A) Internal Nodes

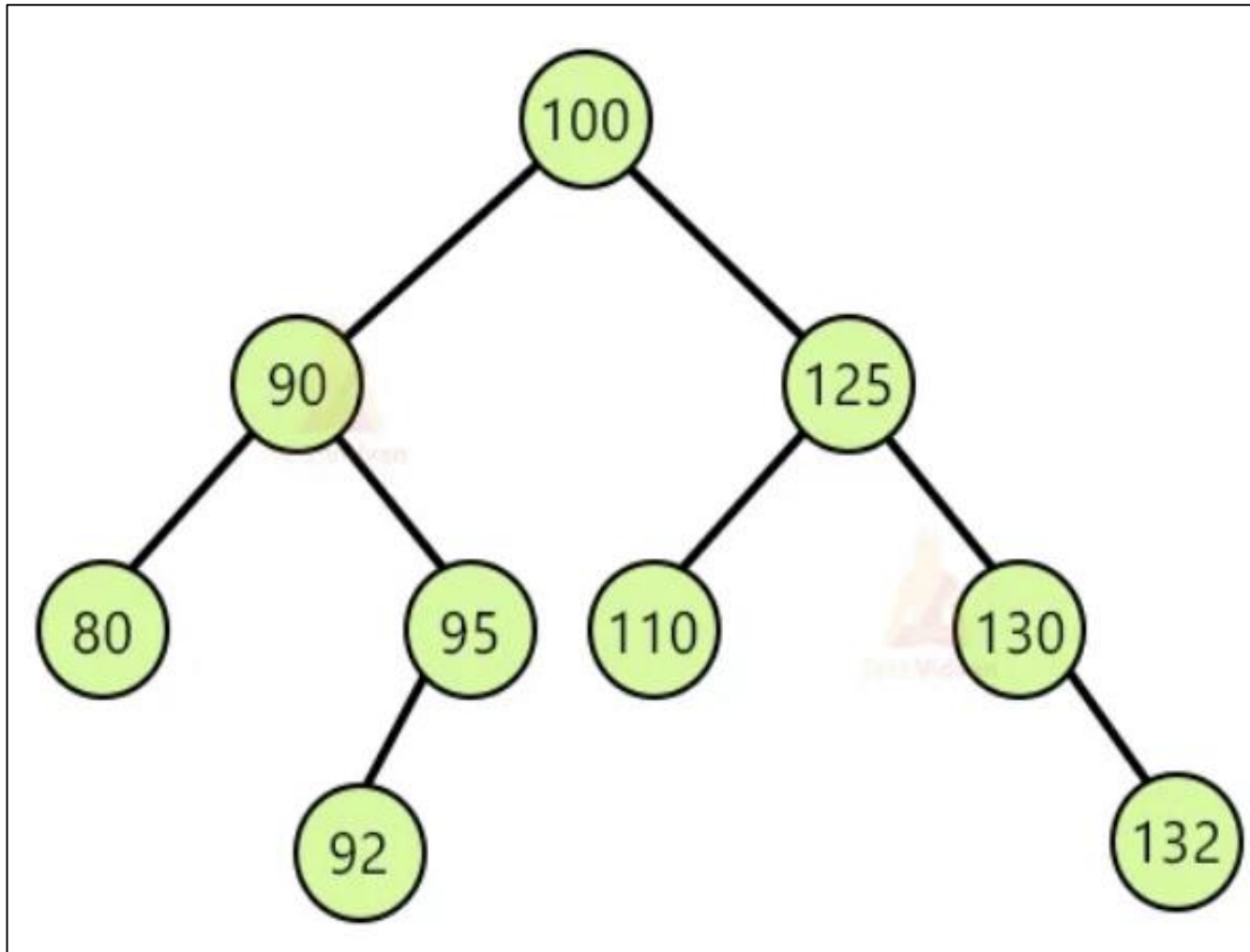


B) External Nodes and Extended BT

Binary Search Tree

- The definition of binary tree which admits duplicates, that is in which each node N has the following property: *the value at N is greater than every value in the left subtree of N and is less than or equal to every value in the right subtree of N .*
- A binary search tree follows some order to arrange the elements.
- In a Binary search tree, the value of left node must be smaller than the parent node, and the value of right node must be greater than the parent node.
- This rule is applied recursively to the left and right subtrees of the root.

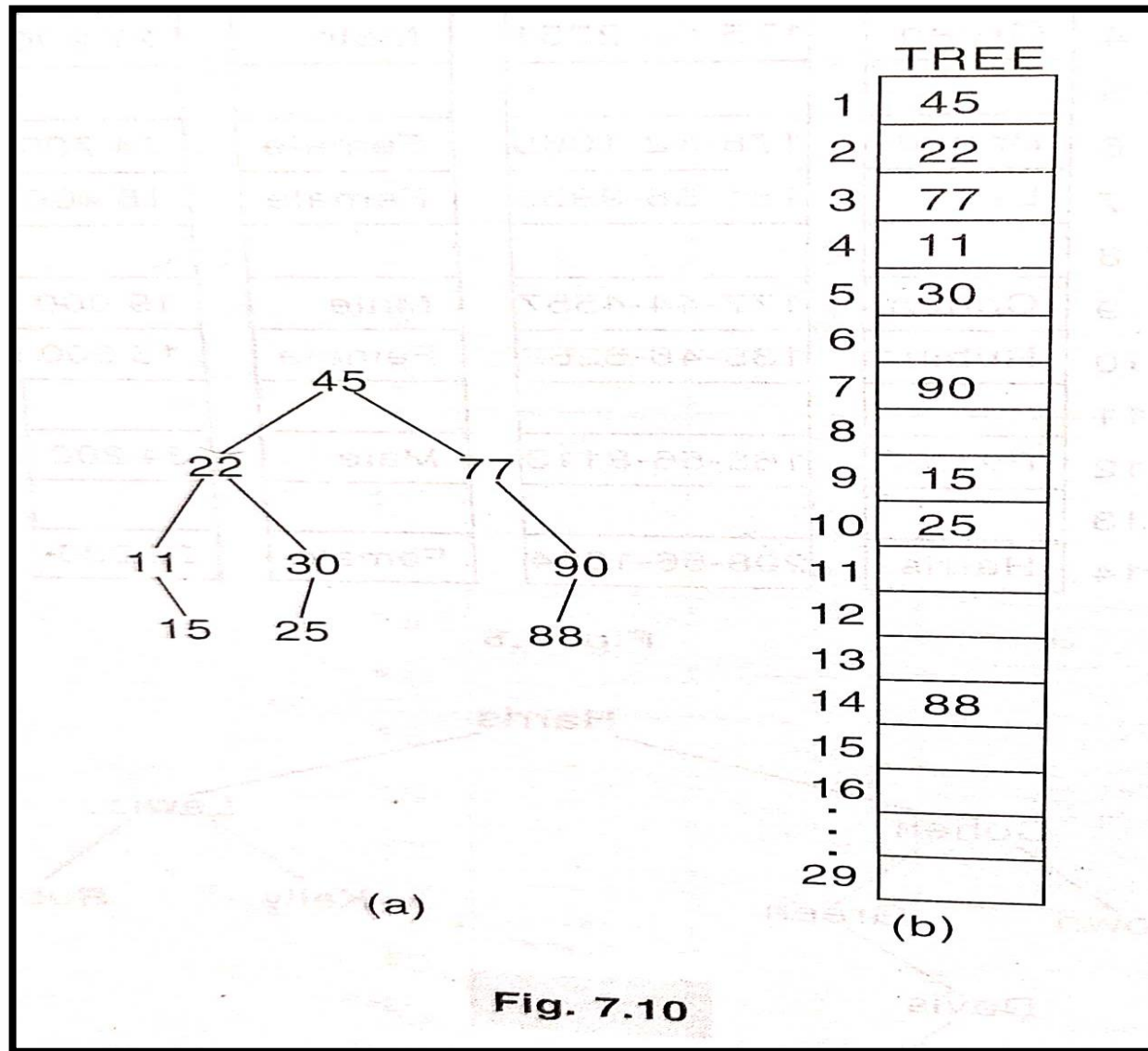
Binary Search Tree



Sequential Representation of BT

- Suppose T is a binary tree that is complete or nearly complete.
- Then there is an efficient way of maintaining T in memory called the *sequential representation of T* .
- This representation uses only a single linear array $TREE$ as follows:
 - (a) The root R of T is stored in $TREE[1]$.
 - (b) If a node N occupies $TREE[K]$, then its left child is stored in $TREE[2 * K]$ and its right child is stored in $TREE[2 * K + 1]$.
- Again, $NULL$ is used to indicate an empty subtree.
- In particular, $TREE[1] = NULL$ indicates that the tree is empty.

Sequential Representation of BT



Sequential Representation of BT

- The sequential representation of the binary tree T in Fig. 7.10(a) appears in Fig. 7.10(b).
- Observe that we require 14 locations in the array TREE even though T has only 9 nodes. In fact, if we included null entries for the successors of the terminal nodes, then we would actually require TREE[29] for the right successor of TREE[14].

Traversing Binary Trees

- There are three standard ways of traversing a binary tree T with root R .
- These three algorithms, called preorder, inorder and postorder are as follows:

Traversing Binary Trees

- **Preorder**

1. Process the root R.
2. Traverse the left subtree of R in preorder.
3. Traverses the right subtree of R in preorder.

- **Inorder**

1. Traverses the left subtree of R in inorder.
2. Process the root R.
3. Traverse the right subtree of R in inorder.

- **Postorder**

1. Traverses the left subtree of R in postorder.
2. Traverses the right subtree of R in preorder.
3. Process the root R.

Preorder Traversal

Algorithm: Initially push NULL onto STACK and then set $PTR := ROOT$. Then repeat the following steps until $PTR = NULL$ or, equivalently, while $PTR \neq NULL$.

- (a) Proceed down the left-most path rooted at PTR, processing each node N on the path and pushing each right child R(N), if any, onto STACK. The traversing ends after a node N with no left child L(N) is processed. (Thus PTR is updated using the assignment $PTR := LEFT[PTR]$, and the traversing stops when $LEFT[PTR] = NULL$.)
- (b) [Backtracking.] Pop and assign to PTR the top element on STACK. If $PTR \neq NULL$, then return to Step (a); otherwise Exit.

(We note that the initial element NULL on STACK is used as a sentinel.)

Example 7.9

Consider the binary tree T in Fig. 7.16. We simulate the above algorithm with T , showing the contents of STACK at each step.

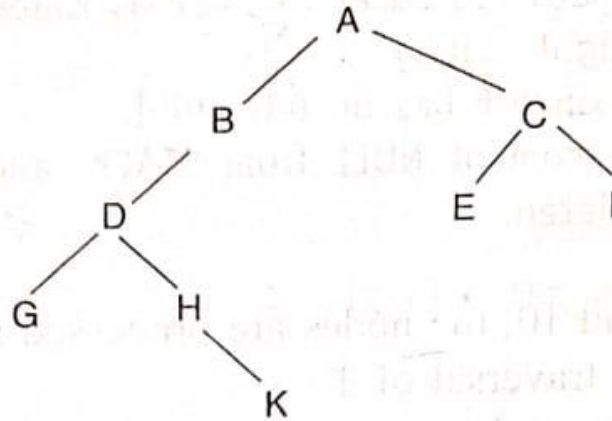


Fig. 7.16

1. Initially push NULL onto STACK:
STACK: $\emptyset$.
Then set $\text{PTR} := A$, the root of T .
2. Proceed down the left-most path rooted at $\text{PTR} = A$ as follows:
 - (i) Process A and push its right child C onto STACK:
STACK: $\emptyset, C$.
 - (ii) Process B. (There is no right child.)
 - (iii) Process D and push its right child H onto STACK:
STACK: $\emptyset, C, H$.
 - (iv) Process G. (There is no right child.)
No other node is processed, since G has no left child.

3. [Backtracking.] Pop the top element H from STACK, and set $PTR := H$. This leaves:

STACK: $\emptyset$, C.

Since $PTR \neq \text{NULL}$, return to Step (a) of the algorithm.

4. Proceed down the left-most path rooted at $PTR = H$ as follows:

(v) Process H and push its right child K onto STACK:

STACK: $\emptyset$, C, K.

No other node is processed, since H has no left child.

5. [Backtracking.] Pop K from STACK, and set $PTR := K$. This leaves:

STACK: $\emptyset$, C.

Since $PTR \neq \text{NULL}$, return to Step (a) of the algorithm.

6. Proceed down the left-most path rooted at $PTR = K$ as follows:

(vi) Process K. (There is no right child.)

No other node is processed, since K has no left child.

7. [Backtracking.] Pop C from STACK, and set $PTR := C$. This leaves:

STACK: $\emptyset$.

Since $PTR \neq \text{NULL}$, return to Step (a) of the algorithm.

8. Proceed down the left most path rooted at $PTR = C$ as follows:

(vii) Process C and push its right child F onto STACK:

STACK: $\emptyset$, F.

(viii) Process E. (There is no right child.)

9. [Backtracking.] Pop F from STACK, and set $PTR := F$. This leaves:

STACK: $\emptyset$.

Since $PTR \neq \text{NULL}$, return to Step (a) of the algorithm.

10. Proceed down the left-most path rooted at $PTR = F$ as follows:

(ix) Process F. (There is no right child.)

No other node is processed, since F has no left child.

11. [Backtracking.] Pop the top element NULL from STACK, and set $PTR := \text{NULL}$. Since $PTR = \text{NULL}$, the algorithm is completed.

The required Preorder Traversal of Tree T

A, B, D, G, H, K, C, E, F

Preorder Traversal

Algorithm 7.1: PREORD(INFO, LEFT, RIGHT, ROOT)

A binary tree T is in memory. The algorithm does a preorder traversal of T , applying an operation $PROCESS$ to each of its nodes. An array $STACK$ is used to temporarily hold the addresses of nodes.

1. [Initially push NULL onto $STACK$, and initialize PTR .]
Set $TOP := 1$, $STACK[1] := NULL$ and $PTR := ROOT$.
2. Repeat Steps 3 to 5 while $PTR \neq NULL$:
3. Apply $PROCESS$ to $INFO[PTR]$.
4. [Right child?]
If $RIGHT[PTR] \neq NULL$, then: [Push on $STACK$.]
Set $TOP := TOP + 1$, and $STACK[TOP] := RIGHT[PTR]$.
[End of If structure.]
5. [Left child?]
If $LEFT[PTR] \neq NULL$, then:
Set $PTR := LEFT[PTR]$.
Else: [Pop from $STACK$.]
Set $PTR := STACK[TOP]$ and $TOP := TOP - 1$.
[End of If structure.]
[End of Step 2 loop.]
6. Exit.

Inorder Traversal

Algorithm: Initially push NULL onto STACK (for a sentinel) and then set $PTR := ROOT$. Then repeat the following steps until NULL is popped from STACK.

- (a) Proceed down the left-most path rooted at PTR, pushing each node N onto STACK and stopping when a node N with no left child is pushed onto STACK.
- (b) [Backtracking.] Pop and process the nodes on STACK. If NULL is popped, then Exit. If a node N with a right child $R(N)$ is processed, set $PTR = R(N)$ (by assigning $PTR := RIGHT[PTR]$) and return to Step (a).

We emphasize that a node N is processed only when it is popped from STACK.

Example 7.10

Consider the binary tree T in Fig. 7.17. We simulate the above algorithm with T , showing the contents of STACK.

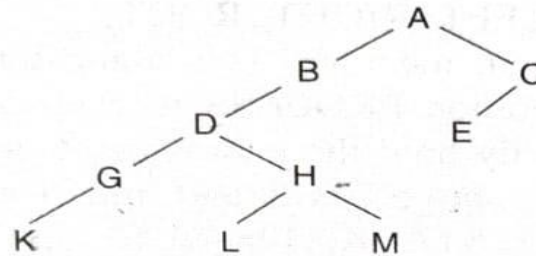


Fig. 7.17

- Initially push NULL onto STACK:
STACK: $\emptyset$.
Then set $\text{PTR} := A$, the root of T .
- Proceed down the left-most path rooted at $\text{PTR} = A$, pushing the nodes A, B, D, G and K onto STACK:
STACK: $\emptyset, A, B, D, G, K$.
(No other node is pushed onto STACK, since K has no left child.)
- [Backtracking.] The nodes K, G and D are popped and processed, leaving:
STACK: $\emptyset, A, B$.
(We stop the processing at D , since D has a right child.) Then set $\text{PTR} := H$, the right child of D .
- Proceed down the left-most path rooted at $\text{PTR} = H$, pushing the nodes H and L onto STACK:
STACK: $\emptyset, A, B, H, L$.
(No other node is pushed onto STACK, since L has no left child.)
- [Backtracking.] The nodes L and H are popped and processed, leaving:
STACK: $\emptyset, A, B$.
(We stop the processing at H , since H has a right child.) Then set $\text{PTR} := M$, the right child of H .
- Proceed down the left-most path rooted at $\text{PTR} = M$, pushing node M onto STACK:
STACK: $\emptyset, A, B, M$.

(No other node is pushed onto STACK, since M has no left child.)

7. [Backtracking.] The nodes M, B and A are popped and processed, leaving:

STACK: $\emptyset$.

(No other element of STACK is popped, since A does have a right child.) Set PTR := C, the right child of A.

8. Proceed down the left-most path rooted at PTR = C, pushing the nodes C and E onto STACK:

STACK: $\emptyset$, C, E.

9. [Backtracking.] Node E is popped and processed. Since E has no right child, node C is popped and processed. Since C has no right child, the next element, NULL, is popped from STACK.

The algorithm is now finished, since NULL is popped from STACK. As seen from Steps 3, 5, 7 and 9, the nodes are processed in the order K, G, D, L, H, M, B, A, E, C. This is the required inorder traversal of the binary tree T.

Algorithm 7.2: INORD(INFO, LEFT, RIGHT, ROOT)

A binary tree is in memory. This algorithm does an inorder traversal of T, applying an operation PROCESS to each of its nodes. An array STACK is used to temporarily hold the addresses of nodes.

1. [Push NULL onto STACK and initialize PTR.]
Set $TOP := 1$, $STACK[1] := NULL$ and $PTR := ROOT$.
2. Repeat while $PTR \neq NULL$: [Pushes left-most path onto STACK.]
 - (a) Set $TOP := TOP + 1$ and $STACK[TOP] := PTR$. [Saves node.]
 - (b) Set $PTR := LEFT[PTR]$. [Updates PTR.][End of loop.]
3. Set $PTR := STACK[TOP]$ and $TOP := TOP - 1$. [Pops node from STACK.]
4. Repeat Steps 5 to 7 while $PTR \neq NULL$: [Backtracking.]
5. Apply PROCESS to INFO[PTR].
6. [Right child?] If $RIGHT[PTR] \neq NULL$, then:
 - (a) Set $PTR := RIGHT[PTR]$.
 - (b) Go to Step 3.[End of If structure.]
7. Set $PTR := STACK[TOP]$ and $TOP := TOP - 1$. [Pops node.]
[End of Step 4 loop.]
8. Exit.

Postorder Traversal

Algorithm: Initially push NULL onto STACK (as a sentinel) and then set $PTR := \text{ROOT}$. Then repeat the following steps until NULL is popped from STACK.

- (a) Proceed down the left-most path rooted at PTR. At each node N of the path, push N onto STACK and, if N has a right child $R(N)$, push $-R(N)$ onto STACK.
- (b) [Backtracking.] Pop and process positive nodes on STACK. If NULL is popped, then Exit. If a negative node is popped, that is, if $PTR = -N$ for some node N, set $PTR = N$ (by assigning $PTR := -PTR$) and return to Step (a).

We emphasize that a node N is processed only when it is popped from STACK and it is positive.

Example 7.11

Consider again the binary tree T in Fig. 7.17. We simulate the above algorithm with T , showing the contents of STACK.

1. Initially, push NULL onto STACK and set $PTR := A$, the root of T :

STACK: $\emptyset$.

2. Proceed down the left-most path rooted at $PTR = A$, pushing the nodes A, B, D, G and K onto STACK. Furthermore, since A has a right child C , push $-C$ onto STACK after A but before B , and since D has a right child H , push $-H$ onto STACK after D but before G . This yields:

STACK: $\emptyset, A, -C, B, D, -H, G, K$.

3. [Backtracking.] Pop and process K , and pop and process G . Since $-H$ is negative, only pop $-H$. This leaves:

STACK: $\emptyset, A, -C, B, D$.

Now $PTR = -H$. Reset $PTR = H$ and return to Step (a).

4. Proceed down the left-most path rooted at $PTR = H$. First push H onto STACK. Since H has a right child M , push $-M$ onto STACK after H . Last, push L onto STACK. This gives:

STACK: $\emptyset, A, -C, B, D, H, -M, L$.

5. [Backtracking.] Pop and process L , but only pop $-M$. This leaves:

STACK: $\emptyset, A, -C, B, D, H$.

Now $PTR = -M$. Reset $PTR = M$ and return to Step (a).

6. Proceed down the left-most path rooted at $PTR = M$. Now, only M is pushed onto STACK. This yields:

STACK: $\emptyset, A, -C, B, D, H, M$.

7. [Backtracking.] Pop and process M, H, D and B , but only pop $-C$. This leaves:

STACK: $\emptyset, A$.

Now $PTR = -C$. Reset $PTR = C$, and return to Step (a).

8. Proceed down the left-most path rooted at $PTR = C$. First C is pushed onto STACK and then E , yielding:

STACK: $\emptyset, A, C, E$.

9. [Backtracking.] Pop and process E, C and A . When NULL is popped, STACK is empty and the algorithm is completed.

As seen from Steps 3, 5, 7 and 9, the nodes are processed in the order $K, G, L, M, H, D, B, E, C, A$. This is the required postorder traversal of the binary tree T .

Algorithm 7.3: POSTORD(INFO, LEFT, RIGHT, ROOT)

A binary tree T is in memory. This algorithm does a postorder traversal of T , applying an operation $PROCESS$ to each of its nodes. An array $STACK$ is used to temporarily hold the addresses of nodes.

1. [Push NULL onto $STACK$ and initialize PTR .]
Set $TOP := 1$, $STACK[1] := NULL$ and $PTR := ROOT$.
2. [Push left-most path onto $STACK$.]
Repeat Steps 3 to 5 while $PTR \neq NULL$:
3. Set $TOP := TOP + 1$ and $STACK[TOP] := PTR$.
[Pushes PTR on $STACK$.]
4. If $RIGHT[PTR] \neq NULL$, then: [Push on $STACK$.]
Set $TOP := TOP + 1$ and $STACK[TOP] := -RIGHT[PTR]$.
[End of If structure.]
5. Set $PTR := LEFT[PTR]$. [Updates pointer PTR .]
[End of Step 2 loop.]
6. Set $PTR := STACK[TOP]$ and $TOP := TOP - 1$.
[Pops node from $STACK$.]
7. Repeat while $PTR > 0$:
 - (a) Apply $PROCESS$ to $INFO[PTR]$.
 - (b) Set $PTR := STACK[TOP]$ and $TOP := TOP - 1$.
[Pops node from $STACK$.][End of loop.]
8. If $PTR < 0$, then:
 - (a) Set $PTR := -PTR$.
 - (b) Go to Step 2.[End of If structure.]
9. Exit.

UNIT-4

CHAPTER-2

GRAPHS

DATA STRUCTURE



Presented By:
Smitha Shetty
Lecturer in BCA Dept

Graph

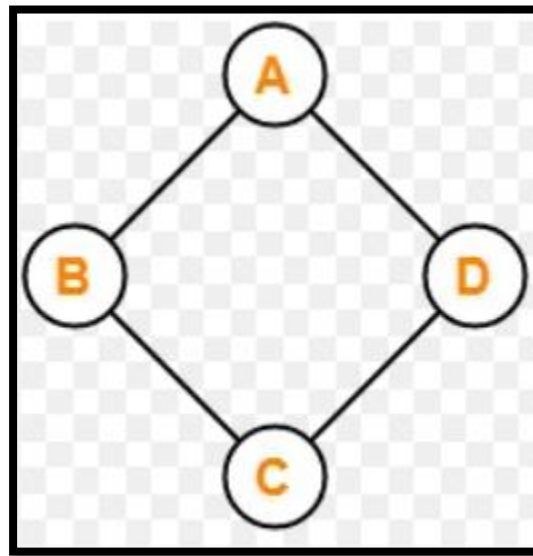
- A Graph consists of two things:
 1. A set V of elements called nodes(or points or vertices)
 2. A set E of Edges such that each edge e in E is identified with a unique pair of nodes in V denoted by $e=[u, v]$.

Graph Terminology

- Suppose $e=[u,v]$, then the nodes u and v are called endpoints of e , and u and v are said to be **adjacent nodes** or **neighbors**.
- The **degree of node u** $\deg(u)$, the number of edges containing u .
- If $\deg(u)=0$, that is, if u does not belong to any edge then u is called an **isolated node**.

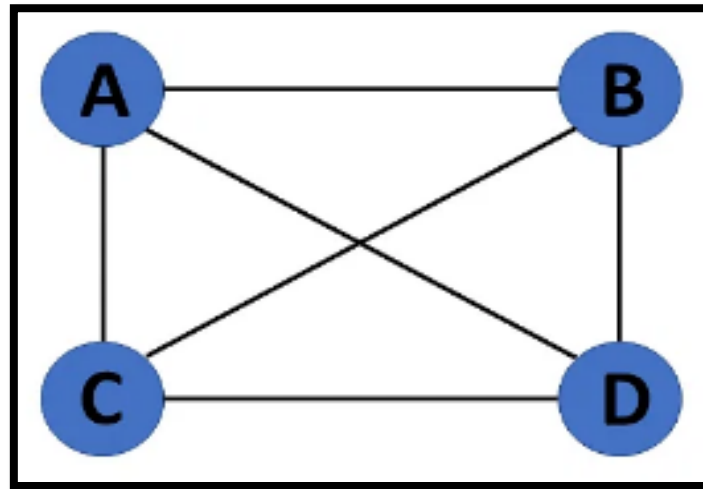
Graph Terminology

- A graph G is said to be **connected**, if and only if there is a simple path between any two nodes in G .



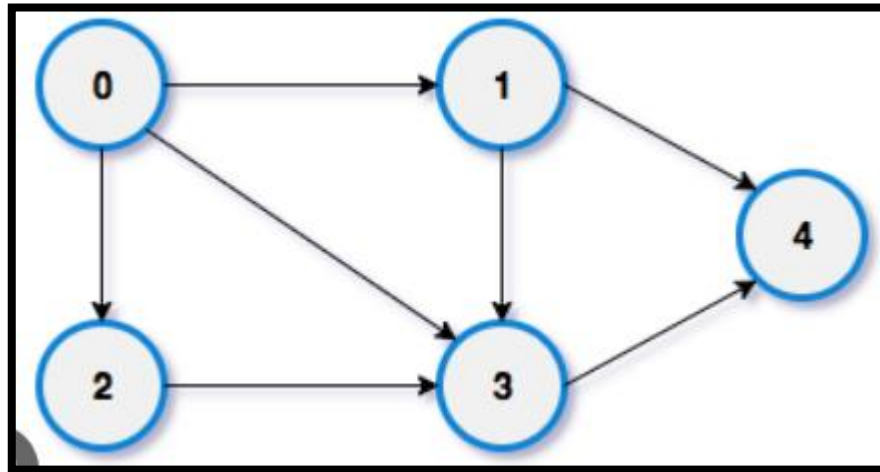
Graph Terminology

- A graph is said to be **complete**, if every node u in G is adjacent to every other node v in G .



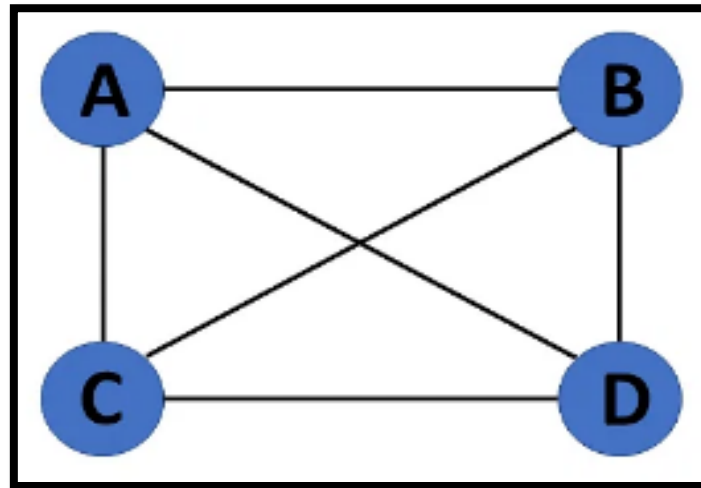
Graph Terminology

- A **Directed** graph G , is also called digraph or graph, in which each edge e in G is assigned a direction.



Graph Terminology

- A **Undirected** graph G , in which edge e in G does not have any direction.



Graph Terminology

- Suppose G is a directed graph with a directed edge $e = (u, v)$.
- Then e is also called an arc.
- Moreover, the following terminology is used:
 1. e begins at u and ends at v .
 2. u is the origin or initial point of e , and v is the destination or terminal point of e .
 3. u is a predecessor of v , and v is a successor or neighbor of u .
 4. u is adjacent to v , and v is adjacent to u .

Graph Terminology

- The **outdegree** of a node u in G , written $\text{outdeg}(u)$, is the number of edges beginning at u .
- Similarly, the **indegree** of u , written $\text{indeg}(u)$, is the number of edges ending at u .
- A node u is called a **source** if it has a positive outdegree but zero indegree.
- Similarly, u is called a **sink** if it has a zero outdegree but a positive indegree.